

AlphaTest

Complete Build Plan & Developer Reference

Python	FastAPI	PostgreSQL	pandas	Streamlit	Plotly
--------	---------	------------	--------	-----------	--------

What is AlphaTest?

AlphaTest is a Python-based quantitative backtesting platform. It supports multiple trading strategies (SMA crossover, RSI), exposes them through a FastAPI REST backend, stores historical OHLCV data and results in PostgreSQL, and visualises everything in an interactive Streamlit dashboard.

System layers

Layer	Technology	Role
Frontend	Streamlit + Plotly	Dashboard: equity curves, trade log, performance metrics
Backend	FastAPI + uvicorn	REST API: accepts strategy configs, runs backtests
Engine	Python + pandas	Signal generation, trade simulation, metrics
Database	PostgreSQL + SQLAlchemy + Alembic	Stores OHLCV data and all backtest results

Get the repo, virtual environment, and database schema in place before writing any business logic. Everything in later phases depends on this foundation.

Folder structure

```
alphatest/  
├── app/  
│   ├── main.py # FastAPI entrypoint  
│   ├── models.py # SQLAlchemy models  
│   ├── schemas.py # Pydantic request/response schemas  
│   ├── database.py # DB connection + session  
│   └── routers/  
│       ├── backtest.py # /backtest route  
│       └── engine/  
│           ├── runner.py # Orchestrates a backtest run  
│           └── strategies/  
│               ├── base.py # Abstract base class  
│               ├── sma.py # SMA crossover  
│               └── rsi.py # RSI overbought/oversold  
│   ├── metrics.py # Sharpe, drawdown, win rate  
│   └── data/  
│       ├── loader.py # Fetches OHLCV from DB  
│       └── ingest.py # Seeds DB with historical data  
├── dashboard/  
│   └── app.py # Streamlit entrypoint  
├── tests/  
│   ├── test_engine.py  
│   └── test_api.py  
├── requirements.txt  
├── .env  
└── docker-compose.yml
```

Step-by-step setup

1. Virtual env	<code>python -m venv venv && source venv/bin/activate</code>
2. Install deps	<code>pip install fastapi uvicorn sqlalchemy psycpg2-binary pandas yfinance streamlit plotly python-dotenv alembic pytest httpx</code>
3. Start Postgres	<code>docker-compose up -d</code>
4. Configure .env	<code>DATABASE_URL=postgresql://user:pass@localhost:5432/alphatest</code>
5. Init Alembic	<code>alembic init alembic → alembic revision --autogenerate -m 'init' → alembic upgrade head</code>

6. Seed data

```
python -m app.data.ingest --ticker AAPL --start 2018-01-01
```

Database tables

- **ohlcv** — ticker, date, open, high, low, close, volume
- **backtest_results** — id, strategy, ticker, params (JSON), created_at, metrics (JSON), trades (JSON)

If you get stuck: The most common Alembic issue is env.py not finding your models. Make sure you import your models inside env.py before calling `target_metadata = Base.metadata`.

Build the engine in complete isolation. Every line of this phase can be tested with pytest before touching FastAPI. This is the most important discipline in the project.

engine/strategies/base.py — abstract base class

```
from abc import ABC, abstractmethod
import pandas as pd

class BaseStrategy(ABC):
    def __init__(self, params: dict):
        self.params = params

    @abstractmethod
    def generate_signals(self, df: pd.DataFrame) -> pd.DataFrame:
        # Must add a 'signal' column: 1=buy, -1=sell, 0=hold
        pass
```

engine/strategies/sma.py — SMA crossover

```
class SMACrossover(BaseStrategy):
    def generate_signals(self, df):
        short = self.params.get("short_window", 20)
        long_ = self.params.get("long_window", 50)
        df["sma_short"] = df["close"].rolling(short).mean()
        df["sma_long"] = df["close"].rolling(long_).mean()
        df["signal"] = 0
        df.loc[df["sma_short"] > df["sma_long"], "signal"] = 1
        df.loc[df["sma_short"] < df["sma_long"], "signal"] = -1
        return df
```

engine/strategies/rsi.py — RSI overbought/oversold

```
class RSIStrategy(BaseStrategy):
    def generate_signals(self, df):
        period = self.params.get("period", 14)
        overbought = self.params.get("overbought", 70)
        oversold = self.params.get("oversold", 30)
        delta = df["close"].diff()
        gain = delta.clip(lower=0).rolling(period).mean()
        loss = (-delta.clip(upper=0)).rolling(period).mean()
        rs = gain / loss
        df["rsi"] = 100 - (100 / (1 + rs))
        df["signal"] = 0
        df.loc[df["rsi"] < oversold, "signal"] = 1
        df.loc[df["rsi"] > overbought, "signal"] = -1
        return df
```

engine/runner.py — trade simulation

```
def run_backtest(df, strategy, initial_capital=10_000):
    df = strategy.generate_signals(df.copy())
    position, cash, trades, equity = 0, initial_capital, [], []

    for i, row in df.iterrows():
        if row["signal"] == 1 and position == 0:
            position = cash / row["close"]
            cash = 0
            trades.append({"date": i, "type": "buy", "price": row["close"]})
        elif row["signal"] == -1 and position > 0:
            cash = position * row["close"]
            position = 0
            trades.append({"date": i, "type": "sell", "price": row["close"]})
        equity.append(cash + position * row["close"])

    df["equity"] = equity
    return df, trades
```

engine/metrics.py — key metrics to compute

- **Sharpe ratio:** $(\text{returns.mean()} / \text{returns.std()}) * \text{sqrt}(252)$
- **Max drawdown:** $(\text{equity} / \text{equity.cummax()} - 1).min()$
- **Win rate:** count profitable trades / total trades
- **Total return:** $(\text{final_equity} - \text{initial_capital}) / \text{initial_capital}$

Testing: Write tests/test_engine.py that loads a small CSV of fake OHLCV data and asserts that signals are generated and metrics are non-null. Run with: pytest. Do this before touching the API.

Expose the engine over HTTP. FastAPI auto-generates a Swagger UI at /docs — use it to test every endpoint before building the dashboard.

schemas.py — request body

```
from pydantic import BaseModel

class BacktestRequest(BaseModel):
    ticker: str
    strategy: str # "sma" | "rsi"
    params: dict
    start_date: str # "2020-01-01"
    end_date: str
    initial_capital: float = 10_000
```

routers/backtest.py — main endpoint

```
@router.post("/backtest")
def run(req: BacktestRequest, db: Session = Depends(get_db)):
    df = load_ohlcv(db, req.ticker, req.start_date, req.end_date)
    strategy = get_strategy(req.strategy, req.params) # factory fn
    result_df, trades = run_backtest(df, strategy, req.initial_capital)
    metrics = compute_metrics(result_df)
    save_result(db, req, metrics, trades) # persist
    return {
        "metrics": metrics,
        "equity_curve": result_df[["close", "equity"]].reset_index().to_dict("records"),
        "trades": trades,
    }
```

main.py

```
from fastapi import FastAPI
from app.routers import backtest

app = FastAPI(title="AlphaTest API")
app.include_router(backtest.router, prefix="/api")
```

Running the server

```
uvicorn app.main:app --reload
```

Open <http://localhost:8000/docs> for the auto-generated Swagger UI — this is your best debugging tool before the dashboard exists.

If you get stuck: A 422 Unprocessable Entity response means Pydantic rejected your request body. Read the detail field in the JSON response — it tells you exactly which field failed validation.


```
fig = go.Figure()
fig.add_trace(go.Scatter(x=eq_df["date"], y=eq_df["equity"], name="Portfolio"))
st.plotly_chart(fig, use_container_width=True)

st.subheader("Trade log")
st.dataframe(pd.DataFrame(data["trades"]))
```

Run the dashboard

```
streamlit run dashboard/app.py
```

If you get stuck: If the Plotly chart doesn't appear, ensure `use_container_width=True` is set. If the API call fails, confirm `uvicorn` is still running and your `.env DATABASE_URL` is correct.

Once the core loop works end-to-end, these are the highest-value additions in recommended order:

Multi-strategy comparison

Accept a list of strategy configs in one request, run each, return all equity curves together. Plot them on a single Plotly chart with a legend. Add a POST /compare endpoint.

Walk-forward testing

Split the date range into in-sample / out-of-sample windows and report performance on each. Prevents overfitting to a single period.

Parameter sweep

Add a POST /sweep endpoint that iterates over a grid of parameter values (e.g. short_window from 10-50 step 5) and returns the best-performing config ranked by Sharpe.

Result history

Add a GET /results endpoint that reads from backtest_results and let the dashboard show a sortable table of all past runs with their metrics.

API key auth

Add a simple API key header check in FastAPI middleware if you plan to expose this beyond localhost.

Quick Reference

Daily commands

```
# Start everything
docker-compose up -d # Postgres on :5432
uvicorn app.main:app --reload # API on :8000
streamlit run dashboard/app.py # Dashboard on :8501

# Database migrations
alembic revision --autogenerate -m "message"
alembic upgrade head

# Tests
pytest tests/ -v

# Seed historical data
python -m app.data.ingest --ticker AAPL --start 2018-01-01
python -m app.data.ingest --ticker SPY --start 2018-01-01
```

Key URLs while running

http://localhost:8000/docs	FastAPI Swagger UI — test endpoints interactively
http://localhost:8000/api/backtest	POST endpoint for running a backtest
http://localhost:8000/api/results	GET endpoint for result history (Phase 5)
http://localhost:8501	Streamlit dashboard

Key discipline reminder

Phase 2 first — get the engine fully working and tested before you touch the API. Every bug is much easier to find in a plain Python script than through an HTTP layer. Run pytest after every strategy change, not just before merging.